

Function Map:

DataBase Functionality:

DataBase: MySQL database hosted on local port;

- `Fetch(username)`: Connects to the MySQL database, takes username parameter, unique identifier for user, and returns the locational information for that account, as well as previous resumes. This keeps in compliance with the ethical requirements provided.
- `Update_term(term, isFirst)`: This function is used to update the term storage for resume data in the database. It takes in the term, some word, or set of words, in the case of the skip-gram, as well as the variable “isFirst.” Which is the variable that keeps track of if this is a new set of terms for the current document being parsed, if so a new entry is created. The function returns the current term count, as well as the document count for the current term, how many documents the current term has appeared in throughout the database.

Positional Functionality:

Dataset: A set containing job positions and descriptions, including skills used;

<https://www.kaggle.com/datasets/adityarajsr/job-descriptions-2025-tech-and-non-tech-roles>

`findPosition.py`:

- `getPos((rawTxt, encodedMap, encodedSkills, model, posset))`: The function has five parameters, `rawTxt`, which is the converted resume into text Strings, the function has a redundancy parsing, which pre-processes the raw text, removes stop words, and un-relevant characters such as punctuation. “EncodedMap,” and “encodedSkills” are two dictionary sets, which implements the Kaggle dataset containing titles, eg: “Cashier”; and descriptions, eg: “handles transactions and purchases.” The function then uses a skip-gram to iterate between all terms, and sets of terms, to find potential job titles within the resume. If the function cannot return a positional title, it then checks the description against the parsed input text, with several weights, and returns the best fit title.
- `Connect(database)`: The function connects to the database agent, and allows this class to access the functions of that agent. Takes in a database title, and returns the connection socket.

- `posSkills(term, encodedSkills, vectorModel)`: Uses the connect function to check if an inputted term is contained in the database, if it is not, the function updates the database. “EncodedSkills” is a dictionary input that maps a position title to its description, which is then used within the skip-gram, to find the closest match, the function returns a dictionary map, final positional guess to its score.

Locational Functionality:

Dataset: A set containing all fifty states, as well as every city within those states, and population per city.

<https://www.kaggle.com/datasets/sergejnuss/united-states-cities-database>

- `findLocation(input1, cities, cityPop, allTF)`: The function takes four parameters, “input1,” which is the raw text output of a resume, cities, which is all cities of each state, weighted by population, or the “cityPop” variable. As well, allTF, which is the term frequency calculated by the database agent. The function iterates through the raw text of the resume, and counts each time a term matches a city name. It uses the population as a weight, the larger the population, the higher the bias is for that term. As well, it uses the term frequency in that specific document, in an effort to prevent common city names from being classified as the final prediction.
- `tfIDF(inputTerm, resumes)`: The function calculates the term frequency-inverse document frequency of all collected resumes in the current session. This is used to help determine if a word is a name, or term, eg: “Commerce, Missouri”; common term, but happens to be a city name.
- `findTF(resume)`: Uses the raw text from the inputted user resume, to determine the term frequency of all terms in that resume. Used to help distinguish between terms and city names.
- `findTfIDF(inputTerm, currentTF, allTF)`: Returns a specific terms inverse document frequency score, used to distinguish between terms and city names. A helper function for `tfIDF()`.

Scraping Functionality:

Dataset: A set containing job positions and descriptions, including skills used;

<https://www.kaggle.com/datasets/adityarajsrj/job-descriptions-2025-tech-and-non-tech-roles>

scraping_Agent.py:

- uploadResume(resPath): Function to upload a resume, determine what extension it is formatted in, eg: “.pdf,” “.txt,” or “.docx.” Returns the extension type, as a string variable, and the individual lines of the resume, as raw strings. Does not pre-process the text.
- termSend(lines): Input parameter is a list of string variables, containing the contents of the user resume. Creates a socket for the database agent, to update terms that are in the user resume, but do not yet exist in the database. Does not return any output.
- encodeTerms(vectorModel): A pre-processing function, which implements the career Kaggle dataset, and pre-embeds all terms in the dataset, to preserve context, and to increase efficiency later in the algorithm, so that terms do not need to be embedded later. Returns a map of embedded vectors.
- encodeSkills(vectorModel): A pre-processing function, which implements the career Kaggle dataset, and pre-embeds all description terms in that dataset, to later be used in the findPosition functions. Returns a map of embedded term vectors to their corresponding job titles, not embedded.

LinkedIn_Agent.py:

- getCity(location): Takes in a String variable, and initializes a String list, a list of four elements, which represent the four cities that LinkedIn uses for data. A second dictionary, which maps cities to their corresponding coordinates. The function computes the coordinates of the input location, and returns a dictionary of all cities mapped to the distance they are from the input location, as well as the minimum distance.
- getData(path, username, password, titles, geoID): Initializes a Playwright context, and accesses the LinkedIn website, using an algorithm admin account, specifically not an admin account for LinkedIn, only an email specifically for the algorithm. The agent function searches the inputted title, and the coordinates for the current city location, and returns the company name, job description, and job title.
- saveRes(results): Saves the output of the getData function to a text file on local storage. Does not return any output.

- `tokenize(text)`: Pre-processing for inputted text. Removes punctuation and stop words, using the NLTK library. Returns all words that were not explicitly removed.
- `build_idf(df)`: Redundant function to calculate the inverse document frequency of an input document.